Foundation for Virtual and Augmented Reality Systems

Prof. Samit Bhattacharya

Department of Computer Science and Engineering

Indian Institute of Technology, Guwahati

Week – 10

Lecture – 51

VR experience optimisation

Hello and welcome to the NPTEL MOOC's course on Foundations for Virtual and Augmented Reality Systems. We are discussing a topic related to the optimization of experience. In particular, we are talking about the optimization of VR navigation experience. So, we will continue our discussion on that topic in this part of the lecture as well. Earlier, we talked about one model for a realistic walking experience; that is, we are assuming that we are having a system-controlled VR tour where user movements and navigation will be controlled by the system. Now, to provide the user a natural walking experience, we have seen a model for that; we can use that model to provide the user the natural walking experience. Now, that model is supposed to display a visual speed, or the optic flow, which is the rate at which the scenes change in the virtual world, so that the user gets the perception of movement. Now, that visual speed is supposed to be provided by the walking model, which matches the user's instantaneous walking speed. Rather, we should say the perceived instantaneous walking speed.

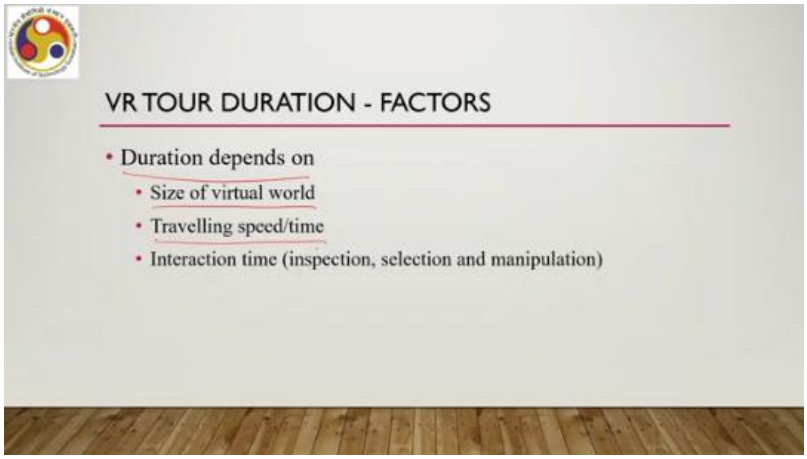
So that is the model. Now, that model itself does not provide anything related to experience optimization. It can only be used to enhance presence in the virtual world. Now, in this lecture, what we are going to do is basically see the use of the model, and with that model, how we can go for an optimization approach where we will try to optimize for cyber sickness.

Also, it may be noted that in this lecture, we are going to talk about one such approach. There are many other approaches. In a subsequent part of the lecture, we will talk about another approach as well. Now, this approach we can broadly term as addressing the cyber sickness concern, that is, optimizing cyber sickness by controlling the exposure duration. So, earlier we talked about one of the factors of cyber sickness being the duration of exposure in the virtual world.

So here, our objective would be to try to optimize the exposure duration so that cyber sickness can be optimized. Details about this can be found at the reference mentioned at the end of the overall lecture, reference number 4. We will point out the relevant reference when we reach the end of the lecture. Now, if you can recollect, cyber sickness is correlated to the exposure duration in a VR environment, as we have repeatedly seen in our discussions on cyber sickness.

Now, the degree of the symptoms rises proportionally with the VR session duration. The more the duration is, the more the exposure is, the more severe the symptoms will be. So, clearly, shorter duration exposure, if possible, can help in getting rid of the cyber sickness problem or at least can help in minimizing the onset of cyber sickness. So, that is the basic assumption on which our approach will be built.

Now, the duration of exposure to cyber sickness depends on several factors, such as the size of the virtual world, the travelling speed, or the travelling time, interaction time, such as interaction with objects, such as inspection, selection, and manipulation of objects. So, there are several factors. The size of the virtual world, let us see how it affects.



Now, a VR environment can be very small, such as a room, or very large, like a city, or sometimes even larger, such as the Majuli Island that we talked about in our case study. For large or larger VR systems or environments, it is quite natural to have a longer duration of VR exposure because the user needs to cover a larger geographical area. Now, we can reduce the exposure time if we can control the speed of movement, the speed of navigation. This is quite obvious: the higher the speed is, the lower will be the time to travel and explore the entire geographical area.

So, duration and travel speed are related, intimately related. Now, duration can be adjusted by adjusting the virtual travel speed, which is quite obvious. So, higher speed effectively refers to a shorter duration of exposure in the virtual world. However, to provide a realistic walking experience, we cannot modify the speed arbitrarily. So,

Teleportation-type movement, of course, would be the best choice. There, the speed is the maximum, so instantaneous movement occurs between two points, so then the overall duration can be reduced. However, that may not always be a good way to approach the problem because teleportation is not a natural movement. Sometimes we look for naturalness. In that case, such type of movement or any arbitrary speed imposed on the user while navigating in the virtual world may not be a good idea.

There can be other ways to control the duration, such as manipulation of the terrain or the or the size of the world. By adjusting the distances between the points of interest in the terrain, by adjusting the terrain of course, we can reduce the travel time and subsequently reduce the exposure time. But the problem here is that it excludes users from getting an

idea about the size of the real space. So, this is similar to having a teleportation-type movement.

If we arbitrarily change the terrain shape, then the user will not get the true feel of the terrain. So, that will hamper the user's ability to build a mental map, which in effect can hamper the user's ability to do the wayfinding activities in the virtual world. To get some idea on this mental map, please refer to our earlier lecture on wayfinding, where we talked about the need for a mental map and how it helps in wayfinding. In the case of a purely system-automated or system-controlled tool, users do not interact. So, there is no scope for the user to interactively change anything.

So, in that case, we can ignore the interaction time, that is, the time required to interact with the virtual objects. In such a scenario, when we are talking of an entirely system-controlled VR tool, the interaction time can be ignored. However, we are concerned with semi-interactive tours. That means we are not totally disregarding interactions.

Users can still interact with objects after a navigation task is complete. So, there the travel will be automatic, system-controlled, but the interactions can happen on a limited scale only at the points of interest, or a small set of objects, all objects that are present in the virtual environment. Only with those objects are users allowed to interact in the virtual world. With other objects, users cannot interact.

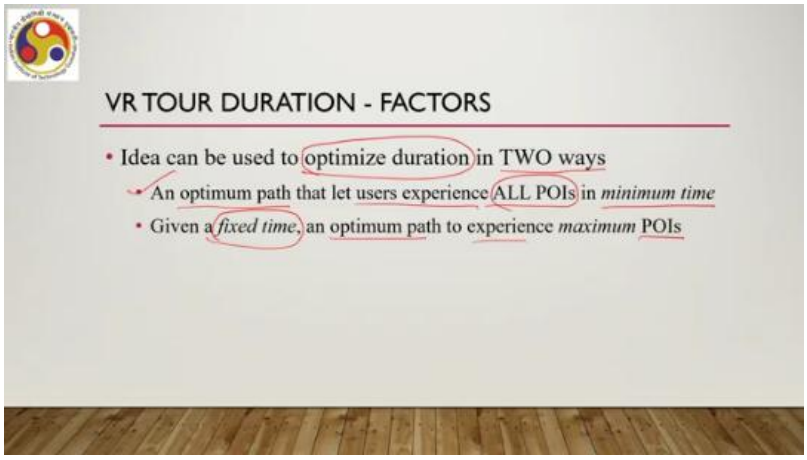
So, if such a scenario is assumed, then we can call it a semi-interactive tour: interaction with POIs and navigation by system control. So, the same idea we have used while formulating the natural walking speed model. So, you can refer to that part of the lecture, the previous lecture, to get some idea on this assumption: that movement from one point to another is controlled by the system tool, whereas at specific chosen points called points of interest, users can interact with those. So, our objective is to deal with semi-interactive system control tools and thereby reduce the duration of exposure to the virtual world.

What can we do? We can limit the interaction time at the points of interest based on an appropriate time. So we have to a priori estimate how much time at the most an average user requires to interact with a point of interest, and accordingly we will allow the user to have that much time at most; after that, the system will automatically force the user to move to another point.

Now, this idea of a semi-interactive tour and appropriate time of interaction—so there is a limit on the time of interaction—can be utilized to optimize the duration of exposure in a virtual tour in either of two ways. One is that we can have an optimum path that lets the users experience all points of interest in minimum time, so the user will be experiencing all points of interest, but the total time required to navigate plus interact will be minimized. Second is, given a fixed time—so we have now a time limit for both navigation and interaction—an optimum path to experience the maximum POIs can be found.

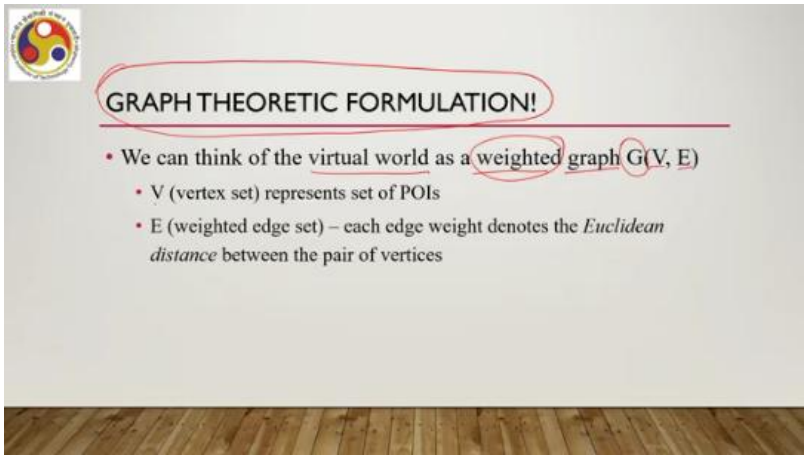
So, in the first case, we have an optimum path that allows the user to interact and explore all points of interest. In the second case, we have an optimum path that allows the users to interact with the maximum possible number of points of interest within a given fixed time. So, with either of these two approaches or problems, we can address how to come up with

an optimum VR tour where the duration of exposure is optimized. And by optimizing the duration of exposure, we are expecting that the prevalence of cyber sickness,



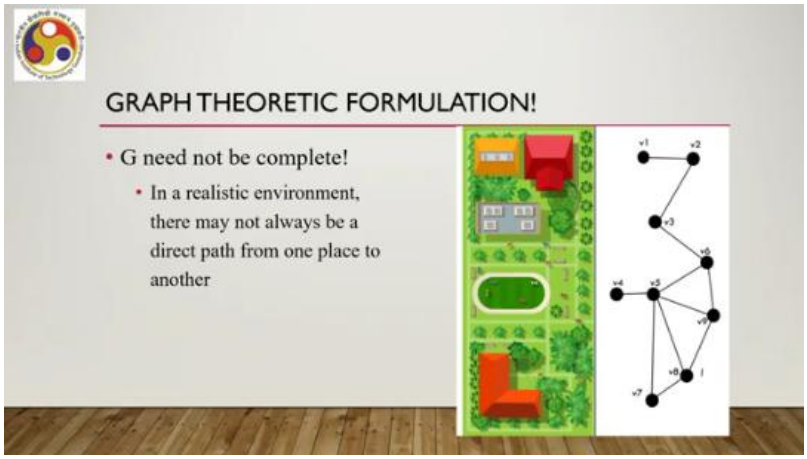
Onset of cyber sickness may be optimized because duration and cyber sickness are directly related. So, let us talk about the first problem, that is, optimum travel path covering all the POIs or points of interest. So, in order to solve this problem, what we need to do first is to formulate the problem in the form of a graph. So, we need a graph theoretic formulation of the problem. So, we can think of the virtual world, that is part of the virtual experience, as a weighted graph.

We can represent it with g. V and E, V is set of vertices, E is set of edges. So, V or the vertex set represents all the points of interest. So, each point of interest we are assuming to be a node or vertex in the graph, and E is the weighted edge set that is the weighted edges between all the POIs.



Now, each edge weight denotes the Euclidean distance between the pair of vertices. So, when we say that we are utilizing weights, we are assigning weights to the edges, that is, the path from one POI to another, the weight we are determining based on the Euclidean distance between these two POIs. Effectively, G represents a top view of the whole virtual world under consideration. So, we need to come up with a G, that is the graph which represents the whole virtual world, a kind of top view of the virtual world.

That is the first thing that we need to do. One important thing here to note is that G need not be complete. So, while going for this graph theoretic formulation, we need not go for finding a graph that is complete. Because in a realistic environment, there may not always be a direct path from one place to another. Now, an example you can see here in this virtual world.

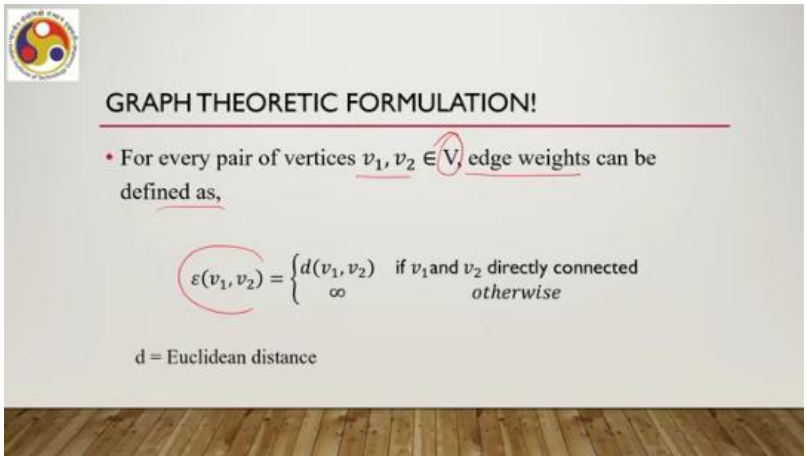


So, on the left-hand side, the colored image is the virtual world having these objects. Several objects are there, and objects within objects can be points of interest. So, a single object does not mean a single node. A single point of interest is a node. So, for example, the swimming pool here is a single object V4.

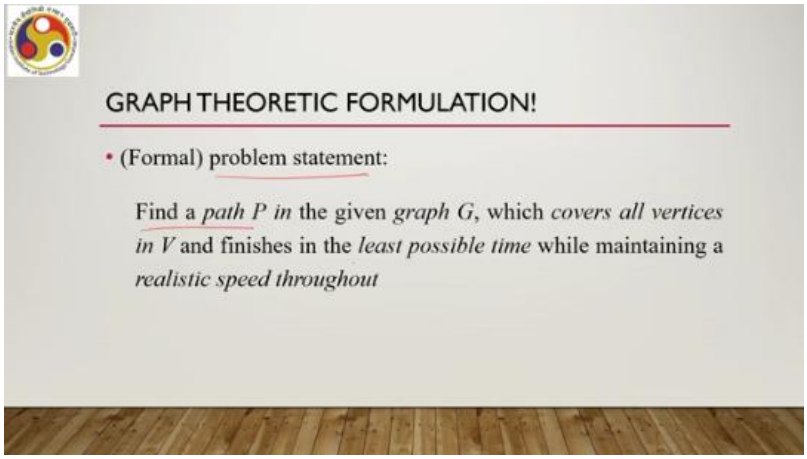
Then it shows that there can be multiple objects from this 1, 2, 3, 4. Then this one can map to this object, this node V3, and each of these can map to these nodes. So, we can map this structure to this node V7, and V8, V9, V6 are some of the points of interest which may be located in this region. So, effectively, although we see that there are 1, 2, 3, 4, 5 structures, there can be more than that number of points of interest; each structure may contain multiple points of interest, and accordingly we have to decide on the nodes and the edges.

Now, in this graph theoretic formulation of this virtual world, you can see that this is not a complete graph because there is no edge from, say, V3 to V4 or V4 to V7 and so on. So, there are many edges, many pairs of vertices where there is no direct path between them. And that is the nature of the problem. In realistic environments, we cannot expect that there will always be a direct path.

So, we can have graphs that are not complete. Now, for every pair of vertices V_1, V_2 belonging to this vertex set V, the edge weights can be defined as the weight ϵ(v_1, v_2) is the Euclidean distance between v_1, v_2 if they are directly connected or a very large number denoted by this infinity symbol if they are not directly connected. So, d is the Euclidean distance.

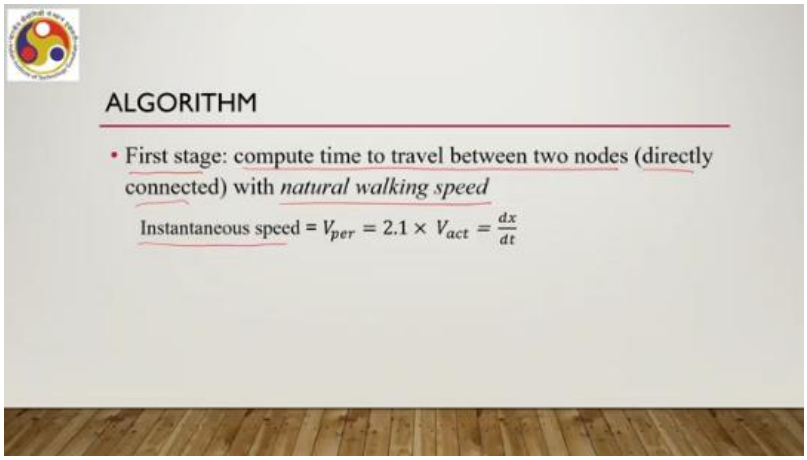


So, if the two vertices are directly connected, then we can compute the Euclidean distance between them and assign that as the edge weight. If they are not directly connected, then we can assume that they are having a weight of a very, very, very large number which we are representing with this infinity symbol. So, given this background, let us formally state the problem. Our problem is to find a path P in the given graph G which covers all vertices in V and finishes in the least possible time while maintaining a realistic speed throughout.

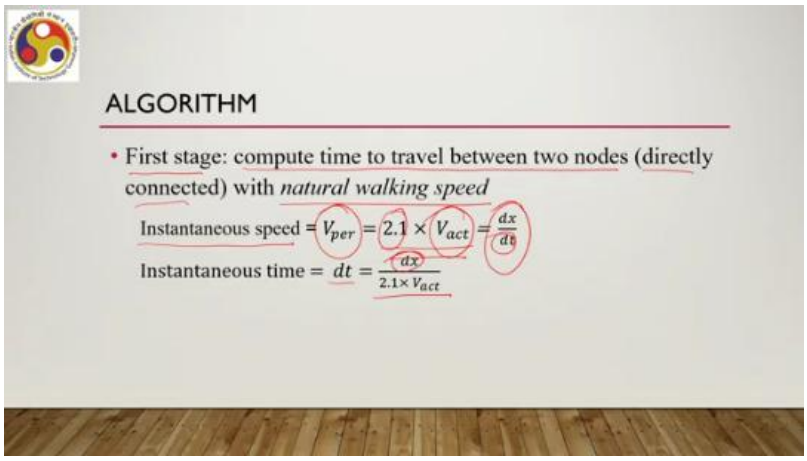


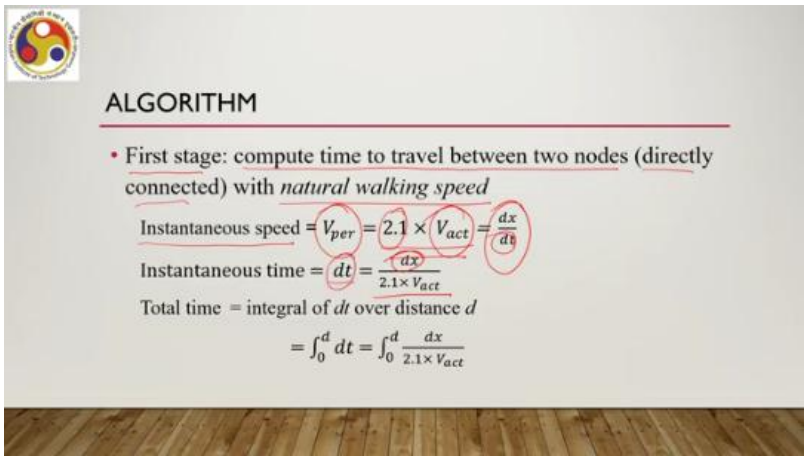
So, once we have formulated the problem in the form of a graph, then we can state the problem as that. We want to find a path within this graph P which includes all the nodes because we are interested in exploring all points of interest. However, while exploring all points of interest, the objective is to minimize the time required to do so while maintaining a realistic speed so that the user gets a realistic feeling. So let us now see how we can come up with an algorithm that solves this problem. So the first stage of the algorithm is to compute the time to travel between two nodes which are directly connected with natural walking speed.

That is the first problem we need to resolve. Now we can make use of our earlier natural walking model to resolve this problem. If you recollect, Earlier we talked about the instantaneous walking speed, which is realistic as represented by V perceived. So here we are talking of perceived instantaneous walking speed, which can be represented in this form 2.1.

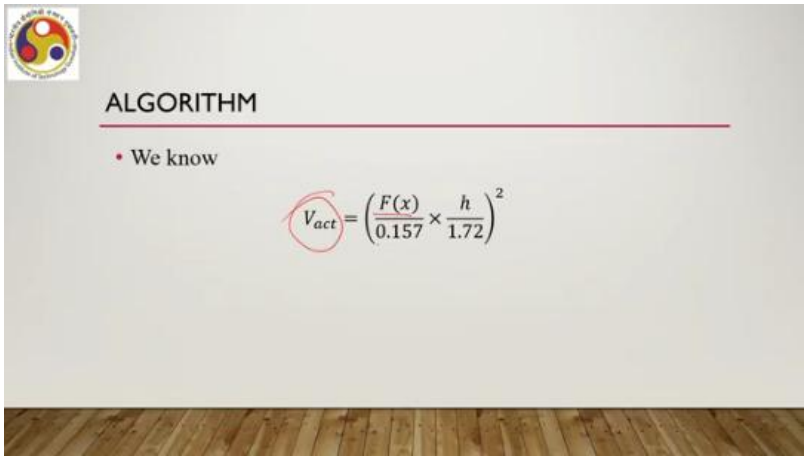


That is the visual gain into the actual instantaneous realistic walking speed V_{act}. Now, this speed, of course, we know from our basic knowledge of calculus that speed can be represented as $\frac{dx}{dt}$. So, then we can say by rearranging the terms that dt, that is this denominator here, is equal to $\frac{dx}{2.1 \times V_{act}}$. The act or the actual instantaneous walking speed which is realistic. So, dt is instantaneous time.

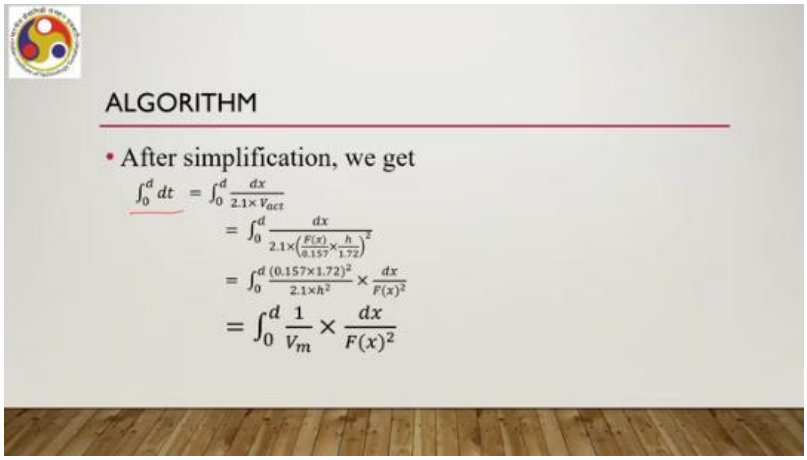




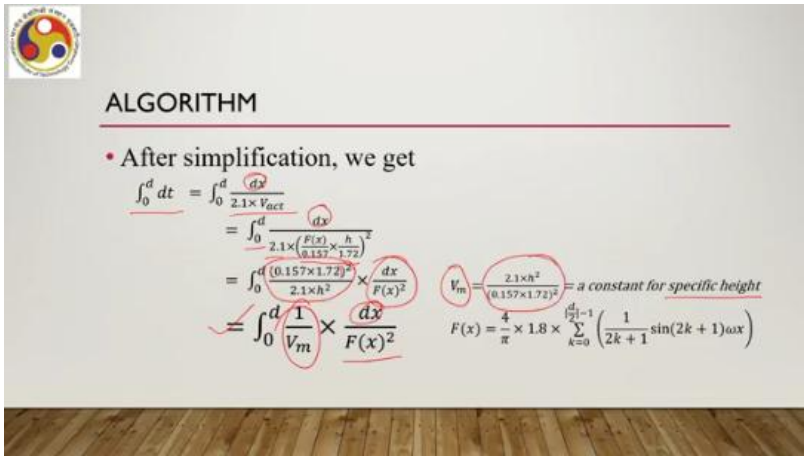
Now, if we want to find out the total time, then we need to take an integral of dt over a distance d. So this is $\int_0^d dt$, or we can say that this is the integration between $\int_0^d \frac{dx}{2.1 \times V_{act}}$. That is the formula using which we can compute the total travel time between two nodes. where the distance between the nodes when they are directly connected is D. We also know that V actual can be represented in the form $\left(\frac{F(x)}{0.157} \times \frac{h}{1.72}\right)^2$



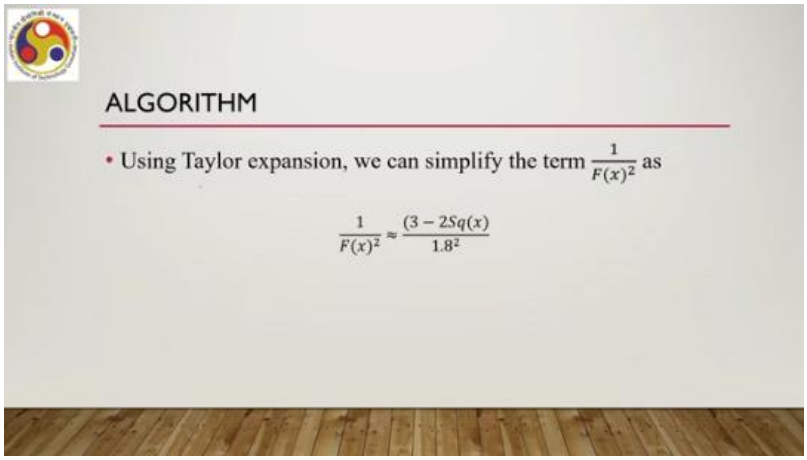
For reference, you can refer to the previous lecture where we talked about these models and terms. Now, f is the instantaneous step frequency and h is the height of the user. So, after simplification, we can get this total time $\int_0^d dt$. That is $\int_0^d \frac{dx}{2.1 \times V_{act}}$, or $\int_0^d \frac{dx}{2.1 \times V_{act}}$, that is $\left(\frac{F(x)}{0.157} \times \frac{h}{1.72}\right)^2$.



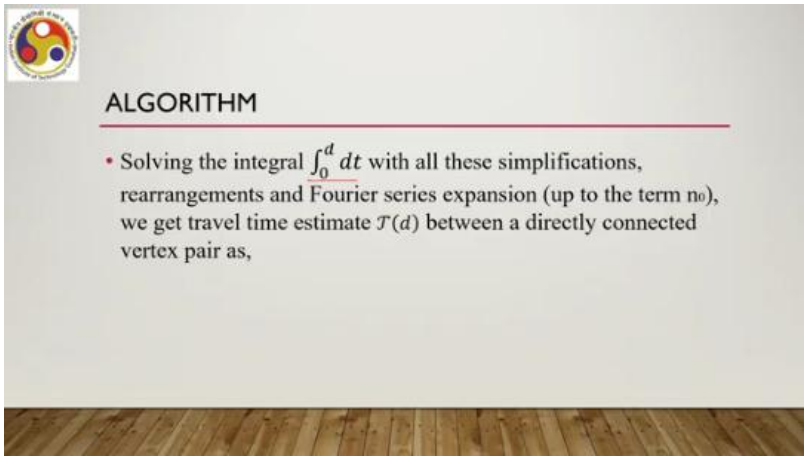
Or by rearranging the terms, we can say that dt equals this term: $\frac{(0.157 \times 1.72)^2}{2.1 \times h^2} \times \frac{dx}{F(x)^2}$. Now, this we can further simplify and rewrite in this form: $\frac{1}{V_m} \times dx/F(x)^2$. Now, here Vm is a constant for a specific height. which is expressed in this way: $\frac{2.1 \times h^2}{(0.157 \times 1.72)^2}$.



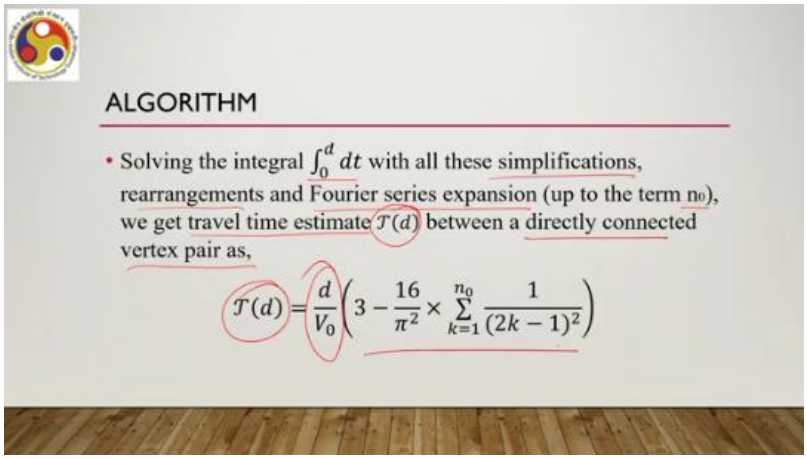
f(x) we can represent using this expression, whole expression, $F(x) = \frac{4}{\pi} \times 1.8 \times \sum_{k=0}^{\left\lfloor \frac{d}{2} \right\rfloor - 1} (\frac{1}{2k+1}\ sin\big((2k+1)\omega x\big))$. So, it is a variable over x where x is the instantaneous point. In fact, this f(x) term we can further simplify using the Taylor expansion.



So, 1 by f(x) square we can simplify as approximately $(3 - 2Sq(x))/1.8^2$, where the $sq(x)$ term is represented by this one, this expression $\frac{4}{\pi} \times 1.8 \times \sum_{k=0}^{\left\lfloor \frac{d}{2} \right\rfloor - 1} (\frac{1}{2k+1}\ sin\big((2k+1)\omega x\big))$. Now, that is the formulation of the time between a pair of nodes which are directly connected. Now, solving the integral dt with all these simplifications, rearrangements, and Fourier series expansions up to the term N_0,

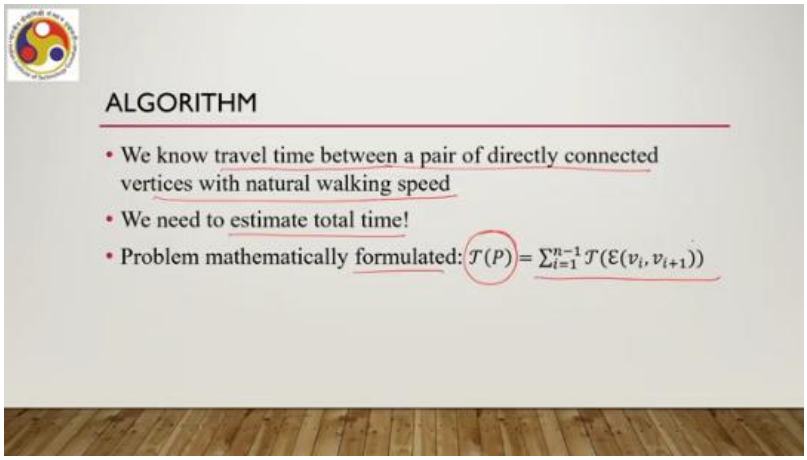


we can get travel time estimate $\tau(d)$ between a directly connected vertex pair as $\tau(d)$ equal to $\frac{d}{V_0}$. and this term $3 - \frac{16}{\pi^2} \times \sum_{k=1}^{n_0} \frac{1}{(2k-1)^2}$. So, this is the result of the computation for dt after using all the simplifications that we have mentioned. So, as you can see, it is a function of d. Now, V_0 is $\frac{2.1 \times h^2}{(0.157 \times 1.72)^2} \times 1.8^2$.

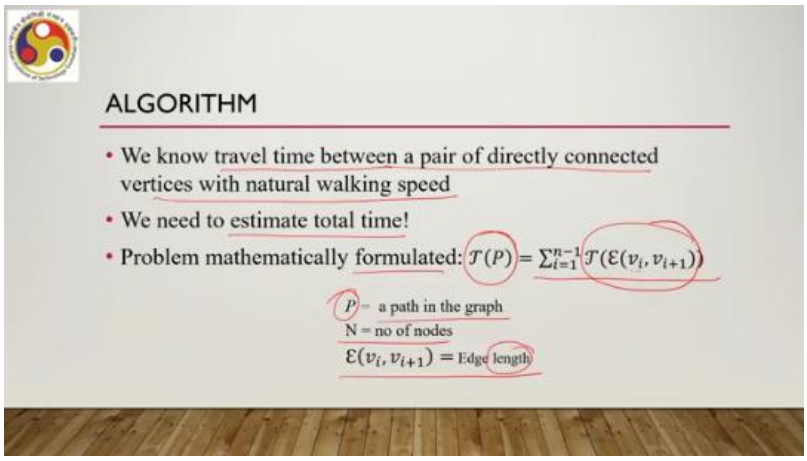


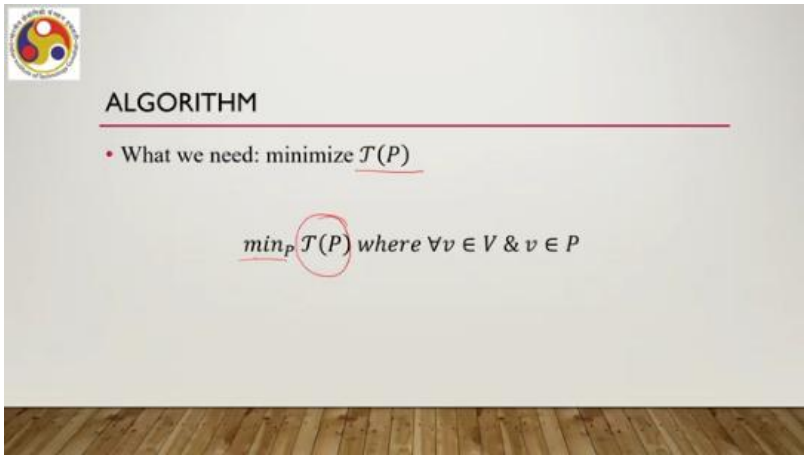
So, this is a constant for a given height. So, if we formulate it in this way, then we can say with certainty that the travel time computed travel time between a pair of directly connected vertices with natural walking speed we can compute. So, our calculation so far assumes this natural walking model, and accordingly we can claim that the time that we are calculating for traveling from one POI to the next, which are directly connected, is the travel time when we are assuming a natural walking speed.

Now, this is between pairs of vertices, and we need to estimate the total time for the entire graph. Now, this problem we can mathematically formulate as total time $\tau(P) = \sum_{i=1}^{n-1} \tau(\epsilon(v_i, v_{i+1}))$. So, we need to find out the times between each pair and then add them up, where P is a path in the graph. So, we are finding out the total time to travel given a path P, and here

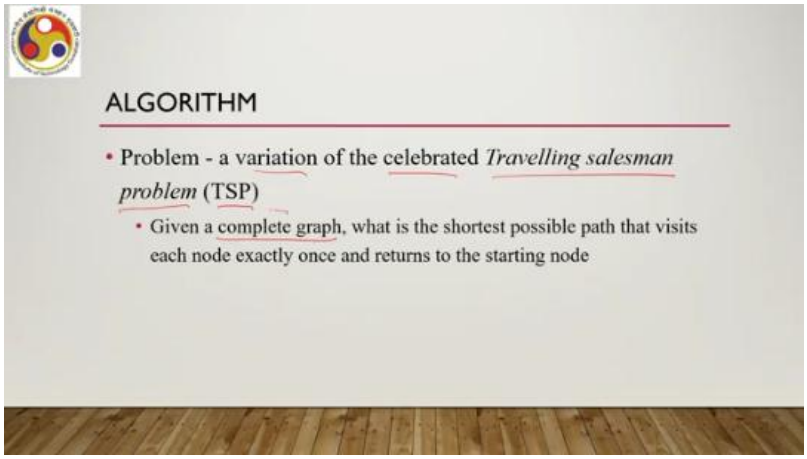


P contains n number of nodes, we are assuming, and then we are calculating the total time to cover the entire path. And this is the edge length we have already seen before; $\epsilon(v_i, v_{i+1})$ is the edge weight or length, the Euclidean distance between the pair of vertices. So, given all such paths, what we need is to minimize this travel time. So, we can recast it as minimize for P given a path where for all V belonging to V and V belonging to P. So, this travel path contains nodes.





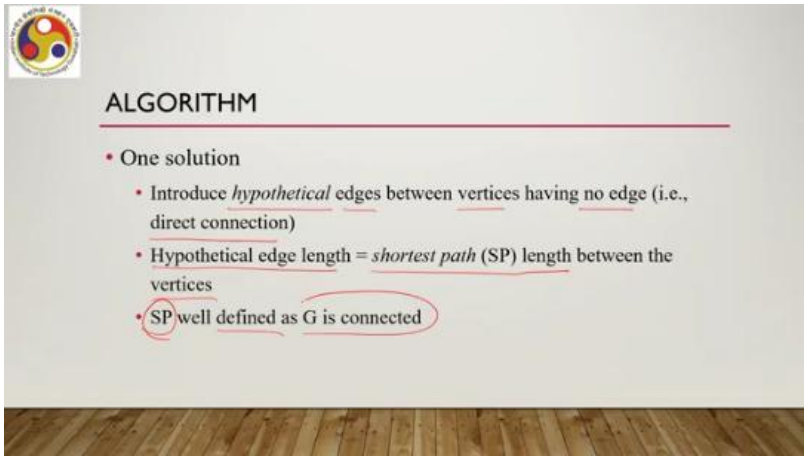
When we minimize, we need to ensure that all the vertices of the graph are included in P. That we have to ensure, and then we have to find out a path satisfying these constraints which gives the minimum duration of travel. Now, this problem is a variation of the classical and celebrated travelling salesman problem, or TSP. Many of us may be familiar with this problem. Which says that, given a complete graph—note the term complete graph—what is the shortest possible path that visits each node exactly once and returns to the starting node?



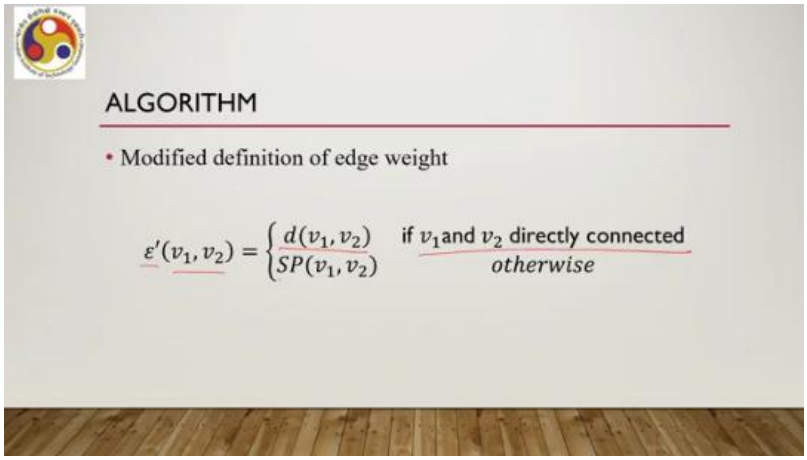
The shortest path is defined based on the total weight of edges in the path. However, in our case, we did not assume that the graph that we are creating is a complete graph. So, there may be cases where there is no Hamiltonian path in G, or in other words, a path where all nodes are covered and each node is visited exactly once, because we are not assuming it to be a complete graph.

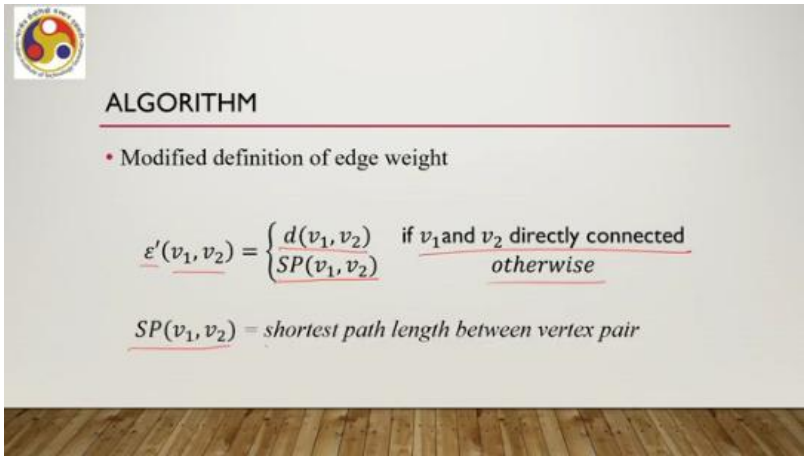
So, one solution to this problem can be to introduce a hypothetical edge between vertices having no edge, or no direct connection. So, when there is no direct connection, we can introduce a hypothetical edge for the sake of computation. And the length of the hypothetical edge is the shortest path length between the vertices. So, if there are two vertices which are not directly connected,

Then we have to identify the shortest path between these two vertices and the corresponding sum of edge weights for the shortest path, and then that edge weight sum of the shortest path will be the weight of the hypothetical edge between these two vertices. Now, the shortest path is well-defined as G is connected. So, there is no issue in finding the shortest path because G is a connected graph. So, then what is the modified definition of edge weight?



If we are assuming the presence of hypothetical edges, let us call this modified edge weight definition to be $\epsilon'(v_1, v_2)$, that is the weight between two nodes v_1 and v_2, which is $d(v_1, v_2)$ if they are directly connected, d is the Euclidean distance, or the shortest path v_1 v_2 if they are not directly connected, that means the sum of the edge weights in the shortest path. Or, in other words, SP here indicates the shortest path length between the vertex pair. So, then we need to revise the problem.



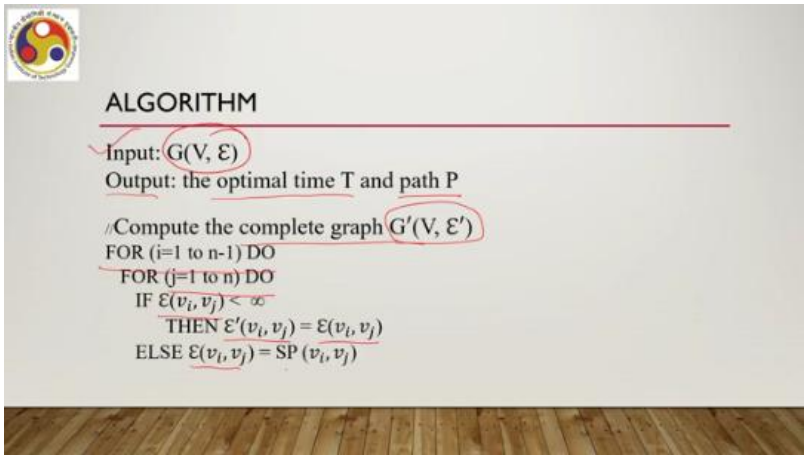


So, we have a modified graph G dash with a modified set of edges V and ϵ dash. Here we are introducing newer edges, hypothetical edges. So, the edge set changes. Now, this modified graph is an instance of the TSP problem, and we need to solve for the optimum path. In order to do that, we can use standard TSP solver approaches or solving algorithms.

We do not need to find a new algorithm, of course, because we are mapping our problem to the standard problem. Now, with this approach of using a standard solver, There is one problem. The output of the TSP solver will give us a path on the graph G' that has an edge set containing hypothetical edges, not on the original graph G where there are no such hypothetical edges. So, what we need to do is to take care of this problem.

So, the solution obtained using a TSP solver may have some hypothetical edges that are present in G' but not in the original graph G. Of course, that is the basic idea. So, we need to replace such edges in the optimal path with the shortest path sp. So, some more work we have to do on the output provided by a standard TSP solver or solving algorithm. We have to modify the output graph output path with replacement of the hypothetical edges in the path with the shortest path. So, with this basic understanding of the algorithm, let us now see the steps of the algorithm. Now, here our input is the original graph G with the vertex set V and edge set ϵ, and the output is the optimal time T and path P. First, we need to compute the complete graph g dash v ϵ dash; that is, we need to compute the hypothetical edge lengths and replace the edge set with an edge set having hypothetical edges so that we can convert the original graph to a complete graph.

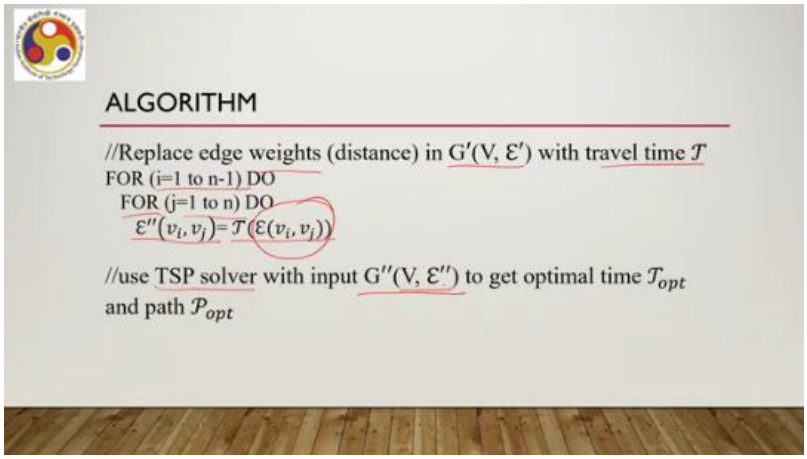
So, for i equal to 1 to n-1, do for j equal to 1 to n do, so n is the total number of nodes. If $\epsilon(v_i, v_j)$ is less than infinity, then $\epsilon'(v_i, v_j)$ equals $\epsilon(v_i, v_j)$. So, if they are directly connected, then there is no need to change anything. Otherwise, you change the edge with the shortest path between the vertices. Next, we need to replace the edge weights or distances in $G'(V, \epsilon')$. with travel time t.

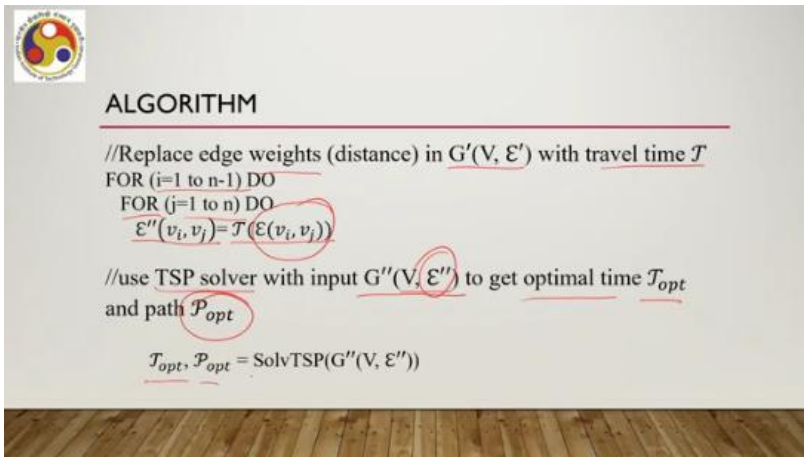


For i equal to again 1 to n, for j equal to 1 to n, for i equal to 1 to n minus 1, j equal to 1 to n, the edge weight between this pair of vertices is the time taken to travel from one vertex to another, which defines the edge. So, first we have modified the graph to a complete graph; then, in order to come up with a travelling salesman problem, we are modifying the edge weights. Earlier it was the Euclidean distance; now we are replacing the edge weights

with the time to travel between the pair of vertices. Once that is done, then we have a complete graph where the edge weights are the time to travel between the pair of vertices defining the edge.

Then we are ready to pass it on to a TSP solver, any standard TSP solver; there are lots available where the input will be the modified graph $G''(V, \epsilon'')$. Now this, modified graph contains edge sets having edges with weights that are travelling time between the pair of vertices, not Euclidean distance or distance between the pair of vertices. So, once we pass it on, we get an optimal time and path as output of the TSP solver. So, let us denote this optimal time as tau opt and the path as p opt. So, then we get tau opt P opt as output of this solved TSP, the TSP solver, where the input is this modified graph.



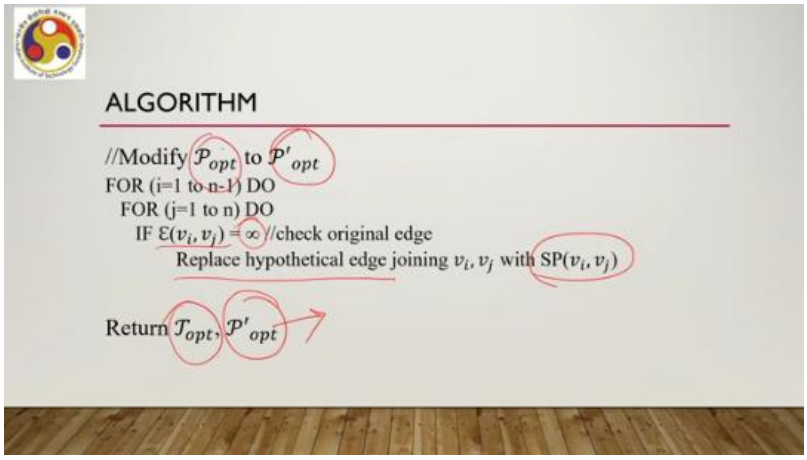


Now, we cannot stop there, as I have already mentioned, because what we get at the end need not be the original graph due to the hypothetical edges. So, we need to modify the output P opt to P dashed opt. so that the hypothetical edges are removed from the output path. So, for all the vertices in the path, what we need to do is basically check in the original edge if there is a direct connection or not. If the original edge contained a very large value, represented by infinity, that means they were not directly connected originally.

So, we have to replace the hypothetical edge between them with the shortest path between them. So, if the original graph contained edges having a very high value, that means they

were not directly connected in the modified graph which was given as input to the TSP solver; we assumed a hypothetical edge. Now, in the path, if such edges appeared between a pair of vertices, then we need to replace them with the shortest path from the original graph. So, once that is done, then we are ready with our solution; that is, we have the optimum time and the optimum path.

This is the path which corresponds to the original graph, whereas the earlier output Popt did not correspond to the original graph because it contained hypothetical edges. So, this is the process to compute the optimal path that takes the minimum time to cover. I hope you could follow the steps of the algorithm and the preceding discussions. However, there is one problem.



This minimum time that we are calculating for this optimal path may still be more than the cybersickness time threshold. So, the algorithm may give a minimum time to cover all vertices in the original graph. However, that time may still be large enough if the terrain, if the virtual environment, is large. So in that case, of course, it may still cross the threshold beyond which there is a possibility of cybersickness. So our whole purpose may be defeated.

So we can, and we have to, do better in such cases. In the next part of the lecture, we will see how we can address this concern when we are dealing with really large geographical terrain with a large number of points of interest. So let us summarize what we have covered so far. So earlier we talked about natural walking speed. We learned a model on that.

Then we talked about how we can optimize our experience by optimizing the possibility of cybersickness, minimizing the possibility of cybersickness by either of two approaches. One is to find out a path in the virtual world that will cover all points of interest, but in a minimum time. And the second one is, given a time threshold, given a time limit, find out a path that will cover the maximum possible points of interest. So, in this lecture, in this part of the lecture, we talked about the first approach, that is, to minimize the time duration when we are trying to cover the virtual world. So, to recap for better understanding, here is what we discussed. So, we converted the problem into a graph theoretic problem. So, we are assuming that we can convert the virtual world into the form of a graph where the nodes are essentially points of interest, the edges between nodes are weighted, and the weight is the Euclidean distance between the points of interest if they are directly connected.

So, the graph need not be a complete graph. Now, we mapped the problem of finding the minimum duration travel within this graph covering all points of interest to a travelling salesman problem, which is a standard classical problem. In order to do so, we first computed the travel time between pairs of vertices. If the pair is directly connected to each other, then we can use the natural walking speed model to compute the time taken to travel from one vertex to another. If they are not directly connected, we introduce some hypothetical edge between them.

which is the shortest path between the pair of vertices, and then compute the travel time from one vertex to another along the shortest path using the natural walking speed. So, in that way, we compute the travel time, which is the new weight we assign to the graph after converting it to a complete graph by introducing hypothetical edges. Now, this modified graph is given as input to the TSP solver. And that provides an output which gives the optimum time and optimal path. Now, that path may contain hypothetical edges.

We need to remove it before we can come up with the final solution. So, all these things were mentioned in the stages of the algorithm. In the next lecture, what we are going to do is talk about the other problem, which is given a time limit. Maximum how many nodes we can cover. with an optimal path.

So essentially the problem is that a time limit is given because whatever we have done with this current approach discussed in this part of the lecture, it can still lead to a large time which may be beyond the threshold beyond which cyber sickness may happen. So we will restrict the time, but given that our objective is to explore the maximum number of points of interest, How to do that? What optimization method can we adopt?

We are going to talk about that in the next lecture. So, in the next lecture, we will be covering the other approach of optimization. So, I hope you could follow the topics. It was a bit involved, I understand, but I hope that my explanations could clear your concepts. So, looking forward to meeting you again in the next lecture.

In the next part of this lecture, we will continue on the same topic. So, thank you, goodbye, and happy learning.